

M002 Extension 2 — Piston & Logic Maze

Topic: Two Pistons + AND Conditions · **Course:** Maze Madness · **Level:** Extension (Advanced) · **Time:** about 60 minutes

This is an **extension challenge** for students who have finished the weekly mazes. The agent follows a **redstone trail** through a long maze. At some junctions it must check **two things at once** (an **AND** condition) to pick the right way. Twice along the path a **lever powers a piston** that pulls a block back and opens the way through. Your job is to get the agent through the **whole maze**, start to finish.

You drive it with **chat commands**: type **l** to turn left, **r** to turn right, **run** to start the solver, and **rl** to teleport the agent back to you.

1 · Read the Chat Commands

Each block of code binds a **chat word** to an action. When you type that word in chat, the agent runs the code.

```
def on_on_chat():  
    agent.turn(RIGHT_TURN)  
    player.on_chat("r", on_on_chat)
```

What does the agent do when you type **r** in chat?


```
def on_on_chat4():  
    agent.teleport_to_player()  
    player.on_chat("rl", on_on_chat4)
```

You typed **rl**. Where does the agent go? When would that help you while solving a long maze?

This maze has two pistons far apart. Why is it useful to test the agent on the *first half* with **l** and **r** before you type **run** on the whole thing?

2 · Trace the Solver 🔍

This is the loop that runs when you type `run`. It follows the trail, makes AND decisions at junctions, and flips a lever when it reaches a piston gate.

```
while True:
    agent.move(FORWARD, 1)
    if agent.detect(REDSTONE, LEFT) and agent.detect(REDSTONE, AHEAD):
        agent.turn(LEFT_TURN)
        agent.move(FORWARD, 1)
    elif agent.detect(BLOCK, AHEAD) and agent.detect(REDSTONE, RIGHT):
        agent.turn(RIGHT_TURN)
        agent.move(FORWARD, 1)
    elif agent.detect(BLOCK, AHEAD):
        agent.interact(AHEAD)
        agent.move(FORWARD, 1)
```

The loop starts with `while True`. When does it stop on its own?

Look at the first check: `detect(REDSTONE, LEFT) and detect(REDSTONE, AHEAD)`. Both have to be true. Describe a spot in the maze where that happens.

The last `elif` is the piston gate. There is a block ahead with no redstone trail beside it. What does `agent.interact(AHEAD)` do there, and why can the agent move forward right after?

Why does the order of the checks matter? What would go wrong if `elif agent.detect(BLOCK, AHEAD)` came *first*, before the two AND checks?

3 · Spot the Bug 🐛

Each block below is broken. Read what it is **supposed** to do, rewrite it so it works, then explain the fix.

Bug A — At a junction the agent should turn left **only when there is redstone on the left *and* redstone ahead**. Right now it turns on either one.

```
if agent.detect(REDSTONE, LEFT) or agent.detect(REDSTONE, AHEAD):  
    agent.turn(LEFT_TURN)  
    agent.move(FORWARD, 1)
```

Hint: "left **and** ahead" — both must be true at the same time.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug B — The solver should keep following the trail for the **whole maze**, checking on every step. This only checks once and stops.

```
agent.move(FORWARD, 1)  
if agent.detect(BLOCK, AHEAD):  
    agent.interact(AHEAD)  
    agent.move(FORWARD, 1)
```

Hint: what wraps a block of code so it repeats until the maze is done?

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug C — At a piston gate the agent must **flip the lever first**, then cross the gap it opens. Here it walks forward before powering the piston, so it hits a closed wall.

```
agent.move(FORWARD, 1)
agent.move(FORWARD, 1)
agent.interact(AHEAD)
```

Hint: power the piston **before** you reach the gap, not after.

Write the fixed code:

Why was it wrong? Why does your fix work?

4 · Make It Your Own

Open the **M002 EXT 2** world and run the solver with `run`. Now **change one thing** and predict what happens before you test it.

Pick **one** idea (or your own):

- Add a new chat command, like `back`, that turns the agent around (two right turns).
- Make the agent place a block (`agent.place(...)`) each time it flips a lever, so you can see how far it got.
- Add an `elif` that handles a junction where redstone is on the **right and ahead** at the same time.

Which change did you pick? Write the code for it.

Predict first: what do you think will happen when you run it?

Then test it. What actually happened? If it surprised you, why?

5 · Finish the Whole Maze 📷

The goal is to get the agent from the **start to the very end** — past both piston gates and every junction. Run `run`, watch where it gets stuck, fix your code, and run it again until it reaches the goal.

When the agent finishes, come back here. Send a photo or video of the agent at the goal, then explain what you did. Use these sentence starters — write 4 to 6 sentences total.

The maze had two pistons, so the agent had to ...

At a junction I used an AND condition to ...

When the agent hit a piston gate, it had to flip the lever first because ...

The change I made in section 4 was ...

The hardest part of finishing the whole maze was ...

If I had more time, I would ...

6 · Record Your Walkthrough 🎥

Take a video on your phone (or a parent's phone) while the agent solves the whole maze. Talk through it like you are teaching someone who has never seen it. Try to use these words: **detect**, **redstone**, **trail**, **AND**, **piston**, **lever**, **loop**.

1. Show the maze and the redstone trail before you start.
2. Type **run** and show the agent following the trail.
3. Read one AND check out loud and say what decision it makes.
4. Show the agent reaching a piston gate and flipping the lever.
5. Show it reaching the goal at the end.

Write what you will say in your video. Plan it here first — you can read from it while filming.

Submit ✓

Send this worksheet + your walkthrough video to teacher on KakaoTalk.